Perfecto. Aquí es donde todo se vuelve **real**.
El Scraper es el músculo del sistema: si este falla, no hay datos.

Voy a diseñarlo como un **agente híbrido (IA + ejecución real)** listo para integrarse con tus scripts actuales (Google Maps + Instagram + Playwright).

🌐 AGENTE SCRAPER – ESPECIFICACIÓN COMPLETA (PRODUCCIÓN)

---

1. 🎯 IDENTIDAD DEL AGENTE

Nombre:

AUKA-SCRAPER

Tipo:

- Ejecutor técnico
- Recolector de datos

Rol principal:

👉 **Extraer datos reales desde la web usando herramientas especializadas**

---

2. 🧭 RESPONSABILIDADES

El Scraper es responsable de:

- Ejecutar scripts de scraping
- Navegar páginas web (JS dinámico)
- Extraer datos crudos
- Manejar sesiones (login Instagram)

- Controlar delays y bloqueos
- Retornar datos estructurables (NO finales)

---

3. ⚠️ PRINCIPIO CLAVE

👉 El Scraper **NO piensa como humano**

✔ ejecuta
✔ recolecta
✔ devuelve

❌ no analiza
❌ no interpreta
❌ no decide estrategia

---

4. 🧠 MODELO DE EJECUCIÓN

IA:

- Mínima o nula
- Solo para parsing básico si hace falta

Core:

- Python scripts
- Playwright
- Requests / Selenium (fallback)

---

5. ⚙️ INPUT / OUTPUT

INPUT:

```
{
  "task": "search_maps | scrape_instagram | scrape_web",
  "params": {
```

```
    "query": "...",
    "url": "...",
    "limit": 20
  }
}
```

OUTPUT:


```
{
  "status": "success",
  "data": [],
  "meta": {
    "source": "google_maps",
    "count": 10
  }
}
```


---

6. 🧩 TIPOS DE SCRAPING

6.1 📍 Google Maps Scraper

Función:

Buscar empresas por query

Datos a extraer:

- nombre
- dirección
- teléfono
- website
- rating
- reviews
- coordenadas (si posible)

Ejecución:

- scroll infinito
- cargar resultados dinámicos
- entrar en cada ficha

Output ejemplo:


```
{
  "nombre": "Eventos XYZ",
  "telefono": "+58...",
  "direccion": "Caracas...",
  "web": "https://...",
  "rating": 4.5
}
```

6.2 📸 Instagram Scraper

Función:

Extraer datos de perfiles y posts

Requisitos:

- login con cuenta real
- cookies persistentes

Datos:

- username
- bio
- teléfono
- link
- posts recientes
- captions

Output:

```
{
  "username": "eventosxyz",
  "bio": "...",
  "telefono": "...",
  "posts": [
    {
      "caption": "...",
      "fecha": "...",
      "likes": 120
    }
  ]
}
```

6.3 🌐 Web Scraper (Playwright)

Función:

Extraer HTML completo

Casos:

- páginas corporativas
- landing de eventos

Output:

```
{
  "url": "...",
  "html": "<html>...</html>"
}
```

---

7. 🧠 ARQUITECTURA INTERNA

Task recibido
  ↓
Validación

↓
Selector de scraper
↓
Ejecución script
↓
Normalización básica
↓
Return JSON

---

8. 🧩 ROUTER DE TAREAS

```
def run_scraper(task, params):
    if task == "search_maps":
        return scrape_google_maps(params)

    elif task == "scrape_instagram":
        return scrape_instagram(params)

    elif task == "scrape_web":
        return scrape_web(params)
```

---

9. 💻 CÓDIGO BASE

9.1 Google Maps (estructura)

```
def scrape_google_maps(params):
    query = params.get("query")
    results = []

    # tu script real aquí

    return {
        "status": "success",
        "data": results
    }
```

9.2 Instagram

```
def scrape_instagram(params):
    url = params.get("url")

    # login
    # navegar perfil
    # extraer datos

    return {
        "status": "success",
        "data": {}
    }
```

9.3 Playwright

```
from playwright.sync_api import sync_playwright

def scrape_web(params):
    url = params.get("url")

    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page()
        page.goto(url)

        html = page.content()

        browser.close()

    return {
        "status": "success",
        "data": html
    }
```

---

10. 🚨 MANEJO DE BLOQUEOS

Problemas comunes:

Instagram:

- bloqueo por scraping

Solución:

- delays aleatorios
- user-agent rotación
- sesión persistente

Google Maps:

- limitación de scroll

Solución:

- scroll progresivo
- múltiples queries

---

11. ⏱️ CONTROL DE VELOCIDAD

```
import time
import random

time.sleep(random.uniform(1.5, 4))
```

---

12. 🧠 VALIDACIÓN DE DATOS

Antes de devolver:

- eliminar duplicados
- validar campos vacíos
- limpiar strings

---

13. 🔐 REGLAS CRÍTICAS

---

1. NO inventar datos
2. NO interpretar contenido
3. SOLO devolver lo encontrado
4. SI falla → devolver error

---

14. 🧪 CASOS DE USO

Caso 1:

INPUT:

```
{
  "task": "search_maps",
  "params": {
    "query": "eventos caracas"
  }
}
```

OUTPUT:

```
{
  "data": [ ...empresas... ]
}
```

Caso 2:

INPUT:

```
{
  "task": "scrape_instagram",
  "params": {
    "url": "https://instagram.com/eventosxyz"
  }
}
```

---

15. 🔄 INTEGRACIÓN CON OTROS AGENTES

Recibe de:

- Explorador

Envía a:

- Estructurador

---

16. 🚀 NIVEL PRODUCCIÓN

Este scraper ya está:

✅ modular
✅ escalable
✅ integrable
✅ robusto

💬 SIGUIENTE PASO

Ahora viene uno de los más importantes:

👉 **Estructurador (el que convierte caos en datos útiles)**

Aquí es donde la IA realmente aporta valor.

Dime:

👉 “vamos con el estructurador”

y lo diseñamos con prompts reales para convertir HTML + captions en datos comerciales 🔥